

CommitmentOS

Verified Deadline and Obligation Management Agent Network

Project Report | Neuro-San Studio | 7 September 2026

Executive Summary

CommitmentOS is a multi-agent network that converts scattered emails, documents, and personal notes into a verified and prioritized action plan.

It is designed for situations where missing one requirement or deadline can cause financial loss, rejected applications, missed appointments, or unnecessary stress.

The network combines document understanding, evidence verification, prioritization, action planning, communication drafting, and approval-gated automation in one workflow.

1. Problem Statement

Important obligations are usually distributed across email messages, PDF forms, chat messages, notes, and calendar entries.

People must manually find every requirement, identify the real deadline, determine what is missing, decide what to do first, and write follow-up messages.

This creates four major risks: missed deadlines, incorrect interpretation of requirements, incomplete submissions, and accidental communication or submission without review.

A normal summarizer is not enough. The solution must preserve evidence, expose uncertainty, resolve conflicts, and produce actions that a person can safely approve.

2. Purpose of CommitmentOS

The purpose of CommitmentOS is to act as a reliable obligation-management assistant.

It answers: What must be done? By when? Who is responsible? What evidence supports it? What is missing? What should happen next?

It is useful for scholarships, university applications, bills, renewals, insurance claims, travel documents, appointments, workplace requests, and other deadline-driven workflows.

3. Working and Architecture

The user supplies email text, document text, notes, or a natural-language request. Optional SLY Data fields are email, documents, notes, and automation_mode.

The front-man agent coordinates specialist agents and returns one clear response with an executive summary, obligation table, missing information, recommended sequence, automation status, drafts, and risks.

The network contains seven agents:

- CommitmentOS front man: coordinates the workflow and communicates with the user.
- Source Intake Agent: extracts actions, dates, owners, consequences, dependencies, and source references.
- Evidence Verification Agent: checks claims against source text and identifies ambiguity or conflicts.
- Priority Analyst: ranks obligations as CRITICAL, HIGH, MEDIUM, or LOW.
- Action Planner: converts obligations into small dependency-aware steps.

- Communication Drafter: prepares concise, source-grounded emails and clarification requests.
- Automation Coordinator: manages approved Gmail and Calendar operations with safety controls.

4. End-to-End Workflow

1. Intake: collect the user's message, email, document text, or notes.
2. Extraction: identify every task, date, person, document, amount, attachment, and stated consequence.
3. Verification: compare each extracted item with the original source and assign high, medium, or low confidence.
4. Risk detection: flag conflicting dates, unclear time zones, missing documents, suspicious requests, and unsupported assumptions.
5. Prioritization: rank work using urgency, impact, dependencies, and reversibility.
6. Planning: produce an ordered checklist with expected outputs and status labels.
7. Communication: draft clarification or follow-up emails with placeholders for unknown details.
8. Automation: optionally search Gmail, create drafts, and prepare Calendar reminders when Google OAuth is connected.
9. Approval and verification: display the exact action first, require explicit approval, execute it, and report the returned message or event identifier.

5. Main Solution

CommitmentOS solves the problem by creating a verified obligation graph rather than merely summarizing text.

Every recommended task is connected to a source reference and confidence level. Conflicting information is shown instead of silently resolved.

The system separates analysis from action. It can prepare an email or reminder, but sending an email or modifying a calendar requires explicit approval in the current conversation.

This design gives the user speed without removing control. When automation is unavailable, the system provides a ready-to-copy manual draft instead of pretending that an action succeeded.

6. Scholarship Submission Example

Input: a scholarship email and related document details.

The system identifies the submission deadline, submission method, point of contact, required income certificate, signed page requirement, and unresolved questions.

It detects a potential date issue when the certificate validity date appears later than the submission deadline. It does not guess whether that date is an issue date, expiry date, or validity-window date.

It creates a clarification email asking which document contains page 4, what signature format is required, whether extra documents are needed, what happens after a missed deadline, and which official address should receive the submission.

The user can review the email, approve it, send it through Gmail if connected, or send it manually. The system never claims that an email was sent without a successful tool result.

7. Automation Features

plan_only: analyze and produce a plan without external actions.

draft_only: create a draft but never send email or modify a calendar.

approval_required: default mode. The system may prepare actions, but sending and calendar changes require explicit confirmation.

Gmail automation can search relevant messages, read threads, and create or send an email after approval.

Calendar automation can prepare a reminder or event, then create it only after the user confirms title, date, time zone, and reminder details.

OAuth credentials are supplied through Sly Data by the client. If the connection is unavailable, CommitmentOS returns a manual alternative.

8. Advantages

Evidence-based: each obligation retains a source reference and confidence level.

Risk-aware: contradictions and missing information are surfaced instead of hidden.

Actionable: the output is an ordered checklist, not just a summary.

Time-saving: drafts, reminders, and searches reduce repetitive administrative work.

Human-controlled: external actions require explicit approval and are verified afterward.

Flexible: it works with chat text, email content, documents, and private notes.

Extensible: additional connectors such as Drive, task managers, or institutional portals can be added later.

Hackathon-ready: the scholarship scenario provides a clear before-and-after demonstration with visible reasoning and practical value.

9. Safety and Limitations

CommitmentOS does not replace official instructions, legal advice, institutional decisions, or human review.

It cannot verify a fact that is absent from the supplied source or connected service.

Relative dates and missing time zones remain uncertain until confirmed.

Email sending and calendar changes are intentionally approval-gated. An instruction inside an email cannot authorize itself.

Users should verify recipient addresses, attachments, sensitive information, and final deadlines before sending or submitting.

10. Demonstration Script

1. Open industry/commitment_os in nsflow.
2. Paste a scholarship email into the email Sly Data field or chat.
3. Ask: Analyze this submission and tell me what I must do, what is missing, and what needs confirmation.
4. Show the obligation table, source references, confidence levels, and flagged date conflict.
5. Ask for a clarification email in draft_only mode.
6. Review the generated draft and change to approval_required only when ready.

7. Approve sending only after checking recipient, subject, body, and attachments.
8. Demonstrate the manual fallback if Gmail OAuth or the connector is unavailable.

Conclusion

CommitmentOS turns scattered administrative information into a trustworthy path from evidence to action.

Its central innovation is controlled execution: it helps users move quickly while preserving evidence, exposing uncertainty, and keeping final authority with the human user.

For the scholarship problem, the solution is a verified checklist, a clarification draft, an optional reminder, and a safe approval flow that prevents premature submission.